Designing a Coding Agent at Scale

Scaler School of Technology | ML System Design and LLMOps | Week 8, Session 4, Bonus Material

Bonus material, Scaler School of Technology. This is not part of the course. A trimmed excerpt covering only **deep-dive 1** (Cache Economics) as a worked example of the rubric. **Practice the remaining four deep-dives on your own** using the ML map and the rubric demonstrated here.

One Instruction, Sixty-Three Model Calls

An engineer types "the auth test is failing, fix it" and presses enter. Forty seconds later a diff appears. In between, the agent read the failing test, searched for the symbol under test, opened four files, formed a hypothesis, edited two of them, ran the suite, read the new failure, corrected its own patch and ran the suite again: roughly **63 model calls and 900,000 input tokens** for one instruction.

This session maps the system that makes such a sequence economically viable. The interesting question is not whether a model can write the patch, since current frontier models can, but what must surround the model. A coding agent is the clearest case in production machine learning of a system where **the cost function and the product are the same object**: every choice that improves quality costs tokens, and tokens are the dominant operating expense.

Why this domain rather than a chatbot. A conversational assistant issues one call per turn against a context the user typed. A coding agent issues tens to hundreds of calls per instruction, against a context that grows with every tool result, over a codebase far larger than any window. It acts on the world by editing files and running commands, it reads untrusted text while doing so, and its outputs become training data for its successor. Inference efficiency, retrieval, agent control and production monitoring all become load-bearing at once, in one product.

Three Properties That Break the Chatbot Mental Model

1

The cost function is the product

A chatbot turn costs a fraction of a cent, so per-request cost is a finance concern rather than an engineering one. An agent working an hour-long task consumes millions of tokens, and the operative business metric becomes **cost per resolved task** rather than cost per call. That metric fights quality directly: more exploration, more files read and more verification passes raise the resolution rate and the bill together. Choosing a larger model does not escape the problem, since it multiplies the same token volume by a higher unit price. Pages 5–7.

2

The system cannot be evaluated directly

There is no label. Establishing whether the agent fixed the bug requires executing the code, and a passing test suite is weak evidence: the agent may have weakened an assertion, special-cased the input, or fixed a symptom. Public benchmarks are saturating, are plausibly contaminated by training data drawn from the same public repositories, and correlate imperfectly with performance on private codebases. This is a sharper version of the offline-versus-online gap familiar from conventional machine learning, because here the offline metric is itself contested. Page 14.

3

The system trains on trajectories it generated

Accepted diffs are the obvious source of preference data. The difficulty is that outcomes are observed only for paths the agent actually chose, so the data carries the fingerprint of the policy that produced it, and acceptance is an unreliable proxy for correctness because engineers approve code they have not read closely. The resulting loop is structurally the classic feedback loop of any deployed decision system: today's policy determines tomorrow's training distribution. Page 19 and cluster G on page 4.

The Token Budget Is the Real Constraint

Why cost grows with the square of the step count

In a conventional request-serving system the binding constraint is latency, and each request is independent. Here the binding constraint is the token budget, and it behaves differently because it **accumulates within a single task**. The table traces one 30-step task under a naive design that resends the full conversation history on every call.

Stage	Input tok	Cumulative	What drives the growth
System prompt + tools	12,000	12,000	Fixed, and identical on every call, which is what makes it cacheable.
Step 1, first search	14,000	14,000	Prompt plus the user instruction.
Step 5	38,000	130,000	Four file reads and two search results now sit in history.
Step 15	94,000	830,000	A test log and several diffs have accumulated.
Step 30	180,000	2,600,000	Near the window limit; compaction is now forced.

Read the cumulative column, not the per-step column. Because each call resends the history, total token consumption over an N-step task grows roughly with N^2 rather than N: doubling the step count approximately quadruples the bill. That single observation explains most of the architecture in this session. Prefix caching makes the repeated portion cheap (page 5), compaction bounds the growth (page 8), and step ceilings exist because an agent that cannot converge is unboundedly expensive (page 4). **A design that ignores the quadratic term will be correct and unaffordable.**

What That Costs, and What It Rules Out

Attaching prices makes the constraint concrete. The figures below are illustrative and dated September 2026; vendors revise them frequently, so the ratios matter more than the absolute numbers.

- **Uncached cost:** 2.6M input tokens at a frontier-tier input price of roughly $3 per million is about $7.80 for *one* task. At 200 engineers running ten tasks a day, that is $\sim \$\mathbf{15{,}600}$ per day, or $4.7M per year. **No internal tool survives that bill**, which is why the naive design is never shipped.
- **With prefix caching:** the repeated prefix is charged once at a write premium and thereafter at roughly a tenth of the read price. Applied to the portion of the 2.6M that is genuinely repeated, this removes the large majority of the spend. The same task lands nearer **$1.00–1.50**, and the annual figure falls by a comparable factor.
- **The lever is layout, not model choice.** A cheaper model reduces the unit price but leaves the quadratic token growth untouched, and it usually raises the step count, which makes the quadratic term worse. Caching attacks the growth itself.
- **Step ceilings bound the tail.** Since cost scales with N^2, a task that fails to converge is not merely unresolved, it is the most expensive item of the day. A ceiling converts an unbounded loss into a bounded one.

Notice what the estimation selected. None of the reasoning above required knowing which model is best at writing patches. The arithmetic alone rules out the naive architecture, mandates prefix caching, and justifies a step ceiling, before any quality comparison is run. **Estimation is not a ritual performed after the design is chosen; it is what chooses the design**. An estimate that shows the obvious approach missing its budget by three orders of magnitude has settled the architecture before any implementation begins.

The ML Map, Part 1 of 2

Clusters A–C | Context, Serving, Agent Loop | Subsystems 1–32

A reference map, not a reading list. Scan it, mark what you could design today, circle what you could not. Deep-dived items are marked ★.

Why five of these get a full design and sixty do not. The five deep-dives are not the five most important subsystems. They are chosen so that each one introduces a **structurally different** problem: cache economics is a case where arithmetic alone selects the architecture (page 5); context assembly is machine learning supplying coefficients to a classical optimiser (page 8); the agent loop is control under uncertainty about one's own progress (page 14); evaluation is a problem with no ground truth (page 17); and injection is an adversary who adapts to the defence (page 20). **Most of the remaining sixty are variations on one of these five.** Speculative decoding (14) and tiered routing (17) are cache economics with different arithmetic; code search ranking (39) is context assembly with a different scoring function; supply-chain checking (57) is the adversarial pattern again. The transferable skill is recognising which of the five a new subsystem resembles, then reusing that design rather than starting from nothing.

A Context and Retrieval, 10 subsystems | what the model is permitted to see

1. **Repo map construction.** A skeleton of the codebase: symbols, signatures and file structure without bodies. Buys a global view at a fraction of the token cost of the source.
2. **AST-aware chunking.** Split on function and class boundaries rather than fixed token windows. A chunk ending mid-function teaches the model a syntax error.
3. ★ **Agentic search versus embedding retrieval.** Deep-dive on page 9.
4. **Embedding index maintenance.** The index is a snapshot, and the agent invalidates it by editing files. Re-indexing cadence trades cost against staleness.
5. ★ **Context compaction and summarisation.** Page 9.
6. ★ **Cache-aware prompt ordering.** Page 6.
7. **Stale-context invalidation.** A file read at step 3 and edited at step 9 sits in history twice with different contents; the model must be told which is current.
8. **Multi-file edit coherence.** A rename must reach every call site. Partial application leaves the repository compiling nowhere.
9. **Dependency-graph traversal.** The transitive blast radius of a change, which bounds what context is genuinely relevant.
10. **History pruning policy.** What to drop first, and what must never be dropped.

B Inference Serving, 12 subsystems | the cost layer

11. ★ **Prefix cache hit-rate engineering.** The single largest cost lever in the system. Page 5.
12. **KV cache memory management.** Block-based allocation so sequences of varying length share memory without fragmentation.
13. **Continuous batching.** Serving many sequences together reuses one weight read across all of them: aggregate throughput rises, and per-user latency rises with it.
14. **Speculative decoding.** A draft model proposes several tokens and the target verifies in one pass. Unusually effective on code, which is highly predictable.
15. **Disaggregated prefill and decode.** Prefill is compute-bound, decode is bandwidth-bound. Agent workloads are prefill-heavy, which makes the split more valuable here than in chat.
16. **Cache-aware routing.** Send a request to the replica already holding its prefix. Round-robin balancing destroys exactly this locality, since it scatters a session across the fleet.
17. **Tiered model router.** A cheap tier for mechanical edits, a frontier tier for planning. The routing decision is itself a prediction problem.
18. **Constrained decoding.** Tool calls must be valid JSON against a schema, enforced at sampling time rather than hoped for and repaired afterwards.
19. **Long-context attention cost.** Attention cost grows with sequence length, so the final steps of a long task are the most expensive per token generated.
20. **Session affinity and eviction.** How long a session's cache blocks survive, and what evicts them under memory pressure.
21. **Autoscaling for bursty agent traffic.** A single user generates sustained multi-call load, so demand is spiky and strongly correlated within a session.
22. **Cache write-cost accounting.** Writes are charged at a premium over reads; the break-even calculation is on page 5.

C Agent Loop and Control, 10 subsystems | what turns a model call into an agent

23. **Tool-call planning and selection.** Which tool, with which arguments, and whether a tool is needed at all.
24. **Subagent spawning.** Delegate a bounded sub-task into a fresh context and return only a summary. A context-management technique as much as a capability one.
25. ★ **Loop and oscillation detection.** Edit cycles that return to a previous state consume budget without progress. Page 12.
26. ★ **Budget enforcement.** Token, wall-clock and step ceilings. Without them, cost per task is unbounded. Page 11.
27. **Interrupt and steering.** The user redirects mid-task without losing accumulated state.
28. **Checkpoint and rollback.** Undoing a bad multi-file edit cleanly.
29. **Parallel tool execution.** Independent reads run concurrently; writes must serialise.
30. ★ **Permission gating.** Which actions require human approval. Page 18.
31. **Retry and error recovery.** When a tool call fails: retry, reformulate, or surrender.
32. ★ **Termination and surrender detection.** An agent reporting partial failure honestly is more useful than one reporting success it cannot substantiate. Page 12.

The ML Map, Part 2 of 2

Clusters D–G | Code Models, Evaluation, Safety, Flywheel | Subsystems 33–65

D Code-Specific Models, 8 subsystems | where the domain asserts itself

33. **Fill-in-the-middle completion versus chat-style edit.** Different training objectives, serving paths and latency budgets, often serving the same product.
34. **Diff application and patch repair.** Models emit diffs that do not apply cleanly. A repair layer that re-anchors the patch recovers a large share of otherwise-failed edits.
35. **Next-edit prediction.** Given the edit just made, where is the engineer going next.
36. **Test generation and test-quality scoring.** A generated test that cannot fail is worse than no test, because it reports false assurance.
37. **Lint and type-error repair loops.** The compiler is a free and exact verifier. Exploiting it well is the cheapest quality gain available.
38. **Commit message and pull-request description generation.**
39. **Code search ranking.** Symbol relevance rather than text similarity.
40. **Language-specific tokenisation and formatting behaviour.** Indentation-sensitive languages fail in ways brace-delimited ones do not.

E Evaluation, 10 subsystems | the layer with no ground truth

41. ★ **LLM-as-judge with bias controls.** Page 15.
42. ★ **Execution-based evaluation harness.** Page 15.
43. **Contamination detection.** Benchmark tasks drawn from public repositories are plausibly present in pretraining data.
44. **Per-language and per-framework slicing.** The aggregate conceals the regression that matters.
45. **Regression suite maintenance.** Frozen tasks, rotated periodically to limit overfitting to the suite.
46. **Human preference collection.** Inter-annotator agreement is low for code-quality judgements even among experienced reviewers.
47. **Accept rate as an online proxy.** Biased upward, because engineers accept code they have not read closely.
48. **Revert rate and follow-up-turn rate.** Lagging signals, and more honest ones.
49. **Trajectory replay for counterfactual comparison.** Run a new policy against a recorded task and hold difficulty constant.
50. **Eval-set hygiene and holdout rotation.**

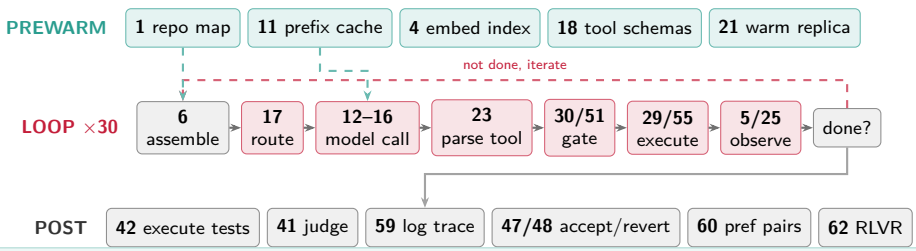
F Safety and Trust, 8 subsystems | the adversarial layer; the counterparty adapts

51. ★ **Prompt injection detection in retrieved content.** Page 17.
52. **Trust-boundary and provenance tracking.** Which tokens arrived from untrusted sources, tracked through the pipeline.
53. **Secret and credential exfiltration prevention.**
54. **Destructive-command gating.** Irreversible operations require a different approval path from reversible ones.
55. **Sandbox hardening.** Bounding what a tool call can reach when it executes.
56. **Network egress policy for tool calls.** Denied by default, allowed by exception.
57. **Supply-chain-aware dependency suggestion.** Typosquatted and abandoned packages are a live recommendation risk.
58. **Licence and provenance checking on generated code.**

G Training Flywheel, 7 subsystems | slow loops that determine the next model

59. **Trajectory collection and storage.** Every session is potential training data, making retention policy a modelling decision.
60. **Preference pair construction.** Derived from accepted and rejected diffs, with all the confounding that implies.
61. **Rejection sampling.** Generate k candidates, keep those that pass execution.
62. **RLVR with execution rewards.** Tests passing is a verifiable reward, which is why coding is one of the few domains with clean reinforcement signal.
63. **Distillation to cheaper tiers.** Moving frontier behaviour onto a tier that can serve inline latency budgets.
64. **Eval-set contamination firewall.** Training data must never touch the eval set; an organisational control as much as a technical one.
65. **Policy-confounding correction.** Importance weighting, or a permanent randomised exploration budget.

Trace One Agent Task: What Runs Once, What Runs Thirty Times



The three lanes have entirely different cost profiles. PREWARM is paid once and amortised across every iteration. LOOP is paid thirty times and dominates the bill. POST is asynchronous and can run on a cheaper model without the user waiting. The instinct to carry away is the one that governs any hot path: **move work out of the hot lane**. When too much happens per iteration, the useful question is not which step to cut, but which can be hoisted into PREWARM or deferred to POST.

Deep-Dive 1: Cache Economics

Subsystems 11 & 22 | The full 10-step rubric | Why prompt layout decides the bill

Step 1 | Clarify the Problem

The objective is cost per *resolved task*, not cost per call. The distinction matters: a cache strategy that lowers per-call cost while raising the step count has made the system worse, and only the task-level metric detects that.

Questions to resolve before designing:

- What cache lifetime is available, and does it span a typical session?
- Are cache writes charged at a premium over ordinary input tokens, and at what ratio?
- Can a session be pinned to the replica holding its prefix?
- Is the tool schema stable for a session, or does it change as capabilities are enabled?

The reframe. This looks like an infrastructure concern and is routinely delegated to a platform team. It is in fact a **prompt-design decision**, because the cache boundary is set by where volatile content sits in the prompt, and that is chosen by whoever writes the prompt.

Step 2 & 3 | Context and Problem Statement

Context. A task runs roughly 30 model calls. Each call resends the accumulated history, so a large and largely *identical* block of tokens appears at the start of every request. Providers expose prefix caching precisely because this pattern is ubiquitous in agent workloads.

Problem statement. Minimise total token spend across a task, subject to a fixed quality bar, by maximising the fraction of each request served from cache. The decision variables are the ordering of prompt segments and the lifetime policy; the constraint is that content the model needs must remain present and current.

Step 4 | Define Requirements

Functional	**What the system must do**
Stable prefix	Hold system prompt, tool schemas and repo map byte-identical for a session.
Correct invalidation	Drop cached content when the underlying file or schema changes.
Prefix-aware routing	Reach the replica already holding the prefix, rather than any replica.
Cost attribution	Report tokens and spend per session, per user and per deploy.
Non-functional	**Target**
Cache hit rate	Above 90% on step 2 onward. Below that, investigate before shipping.
TTFT	Under 2 s at p95, since a cache miss shows up directly as latency.
Observability	Hit rate visible per deploy, with alerting on a drop.
Isolation	No cache content shared across tenants under any circumstance.

Step 5 | Estimations: the Break-Even Calculation

Published pricing at the time of writing charges a cache write at roughly $1.25\times$ an ordinary input token and a cache read at roughly $0.1\times$. Those two ratios settle the design.

- **Cost without caching:** a prefix of P tokens read on n calls costs nP at full price.
- **Cost with caching:** one write at $1.25P$, then $(n-1)$ reads at $0.1P$, giving $1.25P + 0.1P(n-1)$.
- **Break-even:** the cached form is cheaper whenever $1.25 + 0.1(n-1) < n$, which holds for $\mathbf{n \geq 2}$. **Cache anything read more than once.** In a 30-step agent task, the stable prefix is read thirty times.
- **Saving at $n = 30$:** uncached $30P$ against cached $1.25P + 2.9P = 4.15P$, a reduction of roughly $\mathbf{86\%}$ on the repeated portion. Because the prefix dominates token count in a long task, total task cost falls by a similar order.
- **The sensitivity that matters:** these gains assume the prefix survives. One volatile token near the top of the prompt drops the hit rate toward zero and returns the system to the uncached column, at $30P$.

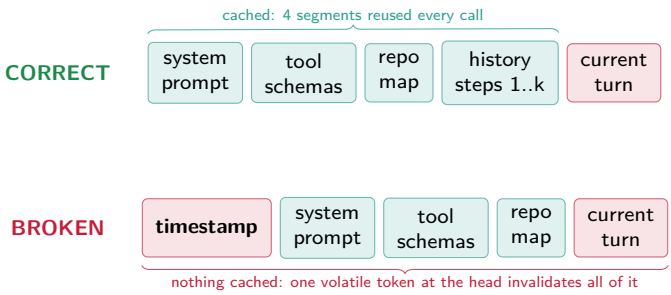
What the arithmetic selected. Agent economics are decided by **prompt layout**, not model selection. That follows from two price ratios and a step count, with no reference to which model writes better patches, and it is why the cache boundary deserves defending as an architectural invariant rather than a tuning parameter.

Cache Economics: Architecture

Order the prompt by volatility | Why the system prompt never changes mid-session

The Central Design Decision: Order the Prompt by Volatility

Caching is **prefix-based**: the provider matches the longest identical leading token sequence. A change at position k invalidates everything from k onward, however small. The rule follows directly: order content by how often it changes, most stable first, so the cache boundary sits as far right as possible.



Why the system prompt never changes mid-session. Inserting a timestamp, a turn counter, or a randomly ordered tool list near the top of the prompt invalidates the whole downstream cache on *every* call. Hit rate collapses, cost rises by roughly the factor computed on page 5, and **nothing functional breaks**. No test fails, no exception is raised, and the output is unchanged, so the regression is invisible to every check except a cost dashboard. This is the most common and most expensive self-inflicted wound in agent design, and it appears again as failure mode 1 on page 23.

Step 6 Decomposition: the Six Sub-Layers

a **Prompt assembly**

Serialises segments in a fixed stable-to-volatile order. This ordering is a system invariant rather than a formatting preference, and should be enforced in code by hashing the assembled prefix and asserting the hash stable across a session.

b **Cache key construction and block hashing**

Content is hashed in blocks and matched block by block from the start. Boundaries matter: a change mid-block invalidates the whole block even when most of its content is unchanged.

c **Prefix-aware routing**

The cache lives in one replica's memory, so a request routed elsewhere misses regardless of what was cached. Round-robin balancing destroys this locality by scattering a session across the fleet; session affinity is the simplest recovery.

d **Eviction and lifetime policy**

Cached blocks occupy memory concurrent sessions also want, so lifetime is a capacity decision: too short and long tasks lose their prefix mid-session, too long and memory is held for sessions that ended.

e **Invalidation on change**

When a mapped file or tool schema changes, cached content is stale. Serving it produces an agent reasoning about a file that no longer exists in that form: a correctness failure, not a cost one.

f **Accounting and attribution**

Per-session and per-deploy telemetry on hit rate, tokens and spend. Without attribution, a cost regression cannot be traced to the prompt change that caused it and is discovered from a monthly invoice.

Cache Economics: Evaluation to Risk

Steps 7–10 | The one experiment in this session that is easy

What is physically being cached. During generation the model stores a key and value vector per token per layer, and the size of that store is $\mathrm{KV} = 2 \times L \times H_{kv} \times d_h \times S \times b$ for L layers, H_{kv} key-value heads of dimension d_h, sequence length S and batch size b. The cache is therefore a **memory-bandwidth artefact**, not a compute one, which is why serving it well is a memory-management problem. Prefix caching exploits the same store *across* requests rather than within one, bounded by the same GPU memory budget.

Step 7 | Evaluation: Offline and Online

Metric	What it tells you, and what it hides
Cache hit rate	The primary health signal. Hides whether cached content was *useful*: a cached but irrelevant repo map scores 100%.
Tokens per resolved task	The metric that matters, combining efficiency with whether the task finished at all.
Cost per resolved task	As above, priced. Hides the distribution: a low mean conceals a tail of runaway sessions.
TTFT, p50 and p95	A miss surfaces as latency before it surfaces as cost, so this is often the earliest signal available.

Step 8 | A/B Testing

Cache changes can be randomised at the **request level**, because they do not alter the output distribution. The same content reaches the model either way; only the billing path and latency differ, so the usual assumptions of a t-test hold.

This is the easiest experiment in the document, and worth naming as such. It is easy for a specific reason: the intervention does not change what the model sees, so it cannot change user behaviour, and requests do not interfere. **Hold on to those conditions.** Page 21 examines an intervention that *does* change the output, and every one of them fails at once.

Step 9 | Deployment and Monitoring

- **Treat cache hit rate as a first-class SLO**, alerting on a drop rather than an absolute threshold. The level varies by workload; a sudden fall does not.
- **Regression-test hit rate in CI.** Hash the assembled prefix and fail the build if it changes unexpectedly, converting an invisible cost regression into a test failure.
- **Attribute cost per deploy**, so a rise in spend can be traced to the release that caused it.
- **Alert on the tail.** Runaway sessions are rare and expensive, moving p99 long before they move the mean.

Step 10 | Risk Management and Trade-offs

Cross-tenant cache leakage

If cache keys derive from content alone, tenants sending identical prefixes may share blocks, and in a coding agent the prefix can contain proprietary source. **This is a confidentiality failure, not an optimisation gone wrong.** Namespace keys by tenant unconditionally.

Serving stale cached context

A cached file body that has since changed leads the agent to reason about code that no longer exists, silently producing confidently wrong patches. Every cached segment needs a validity condition tied to the underlying artefact, not only a time-based expiry.

Vendor cache semantics are not a stable contract

Lifetimes, pricing ratios and minimum cacheable lengths are provider policy and change without notice. Measure the realised saving continuously rather than assuming the figure that justified the design.

Over-fitting the prompt to the cache

Freezing the prefix to protect hit rate discourages improving the system prompt and repo map, both high-leverage quality levers. **Batch prompt changes into versioned releases**, accepting a one-off reset.

This bonus excerpt stops here. Deep-dives 2 through 5, namely Context Assembly Under a Token Budget, The Agent Loop and Control, Evaluation Without Ground Truth, and Prompt Injection in an Agentic Loop, are not included in this copy, and this material is not part of the course. **Use the ML map and the ten-step rubric you just saw worked through once to design the remaining systems on your own.** That repetition, not more reading, is what builds the skill.